

Práctica 2

Comunicación Serie RS232

Alumnos: Francisco Nortes, Mario Martínez.

Profesor: Lenin Lemus.

Índice

1. Objetivo	2
2. Códigos	3
3. Montaje de las conexiones	5
3.1. Cableado y ejecución	5
3.2. Comprobación con el osciloscopio	6
3.3. Amplitud de la señal	8
3.4. Cable NULL-Modem	9
Bibliografía	11

1. Objetivo

El objetivo de esta práctica, al igual que en la anterior, es trabajar con la comunicación serie. En este caso, además del envío de datos mediante el protocolo USB-UART, se utilizará también un protocolo diferente.

Para ello, se configurará la comunicación serie entre dos placas de desarrollo ESP32-S3-DevKitC-1 con el fin de establecer una conexión usando el protocolo RS232. Además, se empleará un cable null modem para intentar obtener el mismo funcionamiento a través de este método.

2. Códigos

Para esta práctica se han usado los mismos códigos que para la anterior como base y se han aplicado unas ligeras adaptaciones.

El código implementa una comunicación serie mediante el protocolo UART en una ESP32, utilizando el puerto secundario **Serial2**, el cual puede emplearse como interfaz compatible con RS232 mediante el hardware de clase.

```
// ESP32 UART2

#define RXD2 17
#define TXD2 18

void setup() {
    Serial.begin(600);          // Debug USB
    Serial2.begin(600, SERIAL_8N1, RXD2, TXD2);

    Serial.println("ESP32 listo");
}

void loop() {
    // Enviar datos
    Serial2.print("S");
    Serial.println("S");

    // Leer si hay datos
    /*if (Serial2.available()) {
        char data = Serial2.read('\n');
        Serial.print("Recibido: ");
        Serial.println(data);
    }*/

    delay(100);
}
```


}

En la función de inicialización (**setup**), se configuran los pines de recepción (RX) y transmisión (TX), asignados a los GPIO 17 y 18 respectivamente. Asimismo, se inician dos canales de comunicación serie: por un lado, el puerto USB (**Serial**) a 600 baudios para la monitorización y depuración desde el ordenador, y por otro, el puerto **Serial2** a la misma velocidad y con configuración estándar de trama (8 bits de datos, sin paridad y un bit de parada).



Durante la ejecución continua del programa (**loop**), la ESP32 envía periódicamente el carácter "A" a través del puerto **Serial2**, con un intervalo de 100 ms entre envíos. Simultáneamente, se comprueba si existen datos disponibles en el buffer de recepción. En caso afirmativo, se lee el dato recibido y se muestra por el puerto serie USB, permitiendo así la visualización de la comunicación entrante.

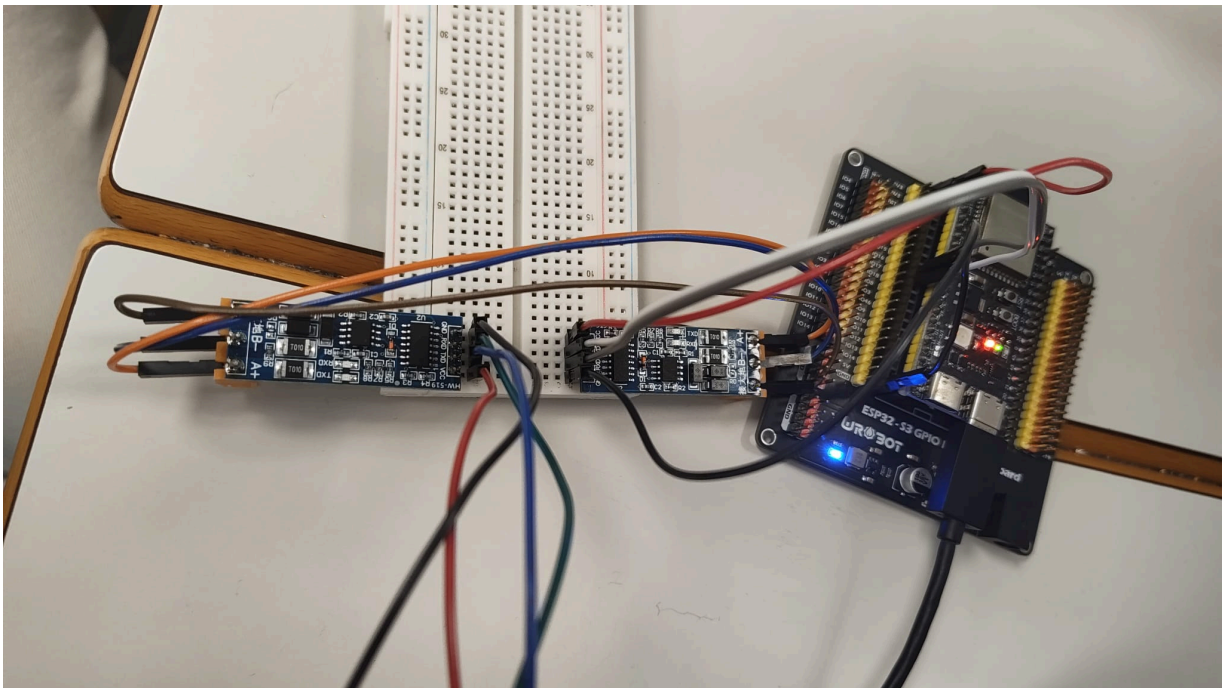
Hemos planteado el código de manera que fuese fácil adaptarlo a las necesidades requeridas. Esto es que el mismo código sirve para enviar o recibir. Simplemente hay que comentar la parte del código que no se va usar.

```
/*if (Serial2.available()) {  
    char data = Serial2.read('\n');  
    Serial.print("Recibido: ");  
    Serial.println(data);  
}*/
```

3. Montaje de las conexiones

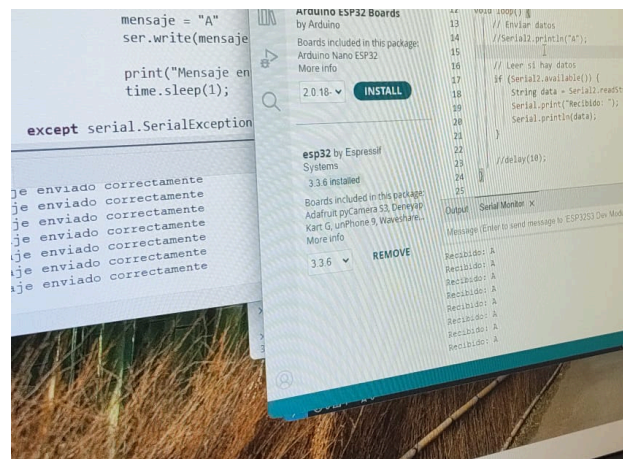
3.1. Cableado y ejecución

Para realizar el montaje, se conectó el pin TXD2 de la primera placa con el pin RXD2 de la segunda, y el pin RXD2 de la primera con el pin TXD2 de la segunda. Además, se unieron las masas de ambas placas para asegurar una referencia común tal y como se muestra en la figura:



Una vez completadas las conexiones se procedió a ejecutar los programas en cada una de las ESP32-S3-DevKitC-1, iniciando así el envío del flujo de mensajes "S".

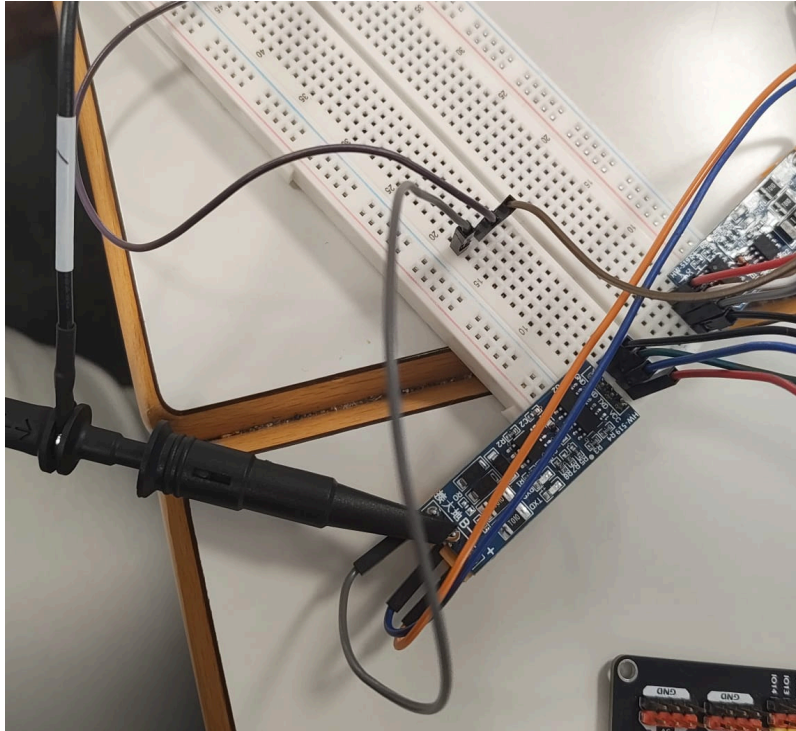
En la imagen se puede observar que el sistema funciona correctamente, ya que la comunicación entre ambas placas se realiza sin problemas.



3.2. Comprobación con el osciloscopio

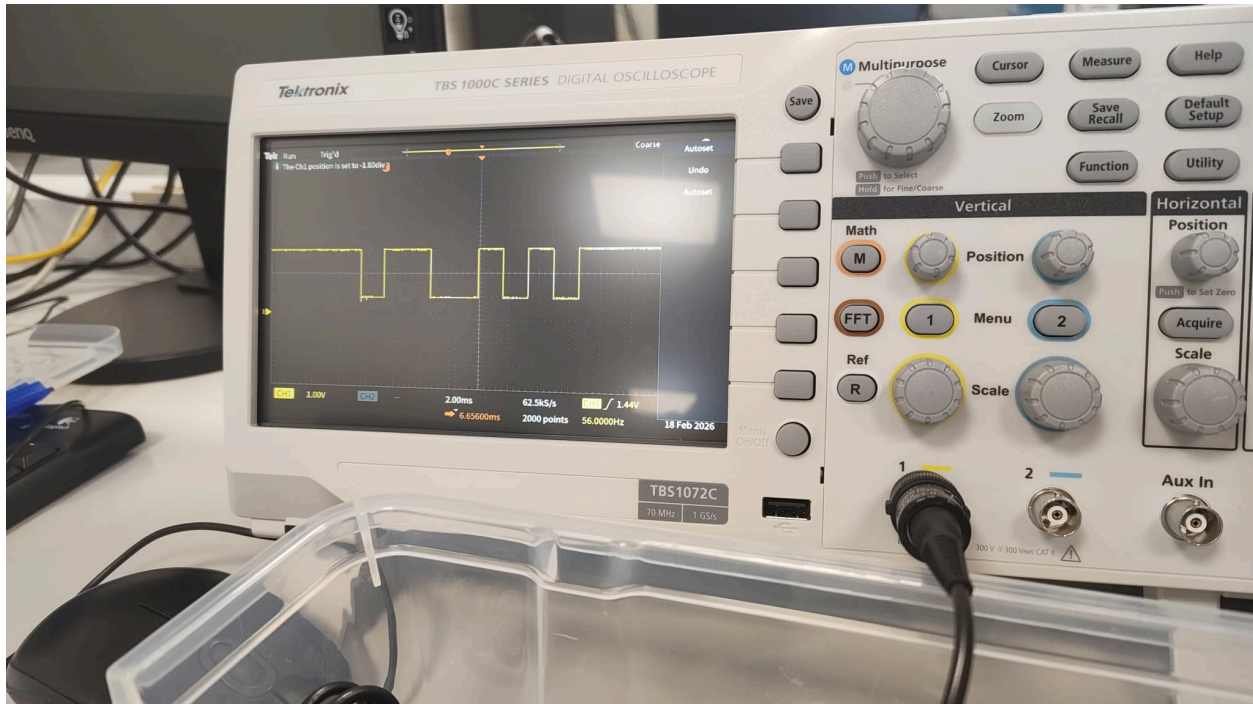
Para comprobar que la comunicación funcionaba correctamente a nivel de señal, pasamos a la comprobación con un osciloscopio para verificar que el carácter "S" se transmitía de forma continua.

En primer lugar, se conectaron las sondas del osciloscopio a los pines de RX y TX del sistema RS232, lo que permitió observar directamente la señal eléctrica asociada a la transmisión de datos.



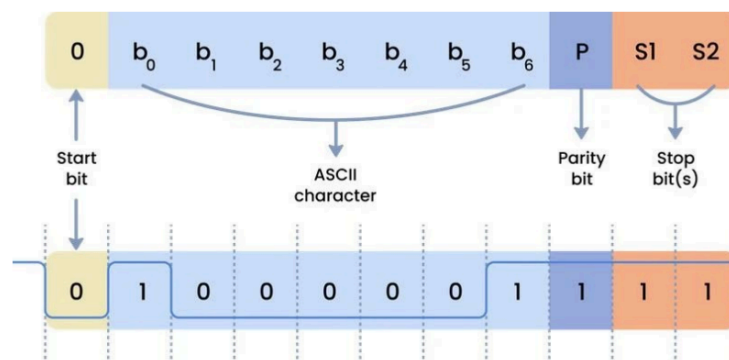
Tras realizar varios ajustes en el equipo haciendo uso del "Autoset", se consiguió visualizar de forma clara la forma de onda correspondiente al envío repetido del carácter "S".

De este modo, se pudo confirmar no solo que la comunicación estaba funcionando, sino también que la señal se transmitía de manera estable y continua.



El carácter “S” en código ASCII tiene un valor decimal de 83, que corresponde a 0x53 en hexadecimal y a **01010011** en binario.

Como se está usando RS-232, los bits no se envían directamente tal cual, sino que se organizan en una trama. Esta comienza con un bit de inicio con valor 0, seguido de los 8 bits de datos, y finaliza con uno o varios bits de parada con valor 1. En este caso concreto no se utiliza bit de paridad, por lo que la estructura es más sencilla.



Además, hay que tener en cuenta que a la hora de enviar, se invierte el orden de la trama de bits, de manera que el LSB se envía primero.

Si el byte de 'S' es (MSB → LSB): **01010011**

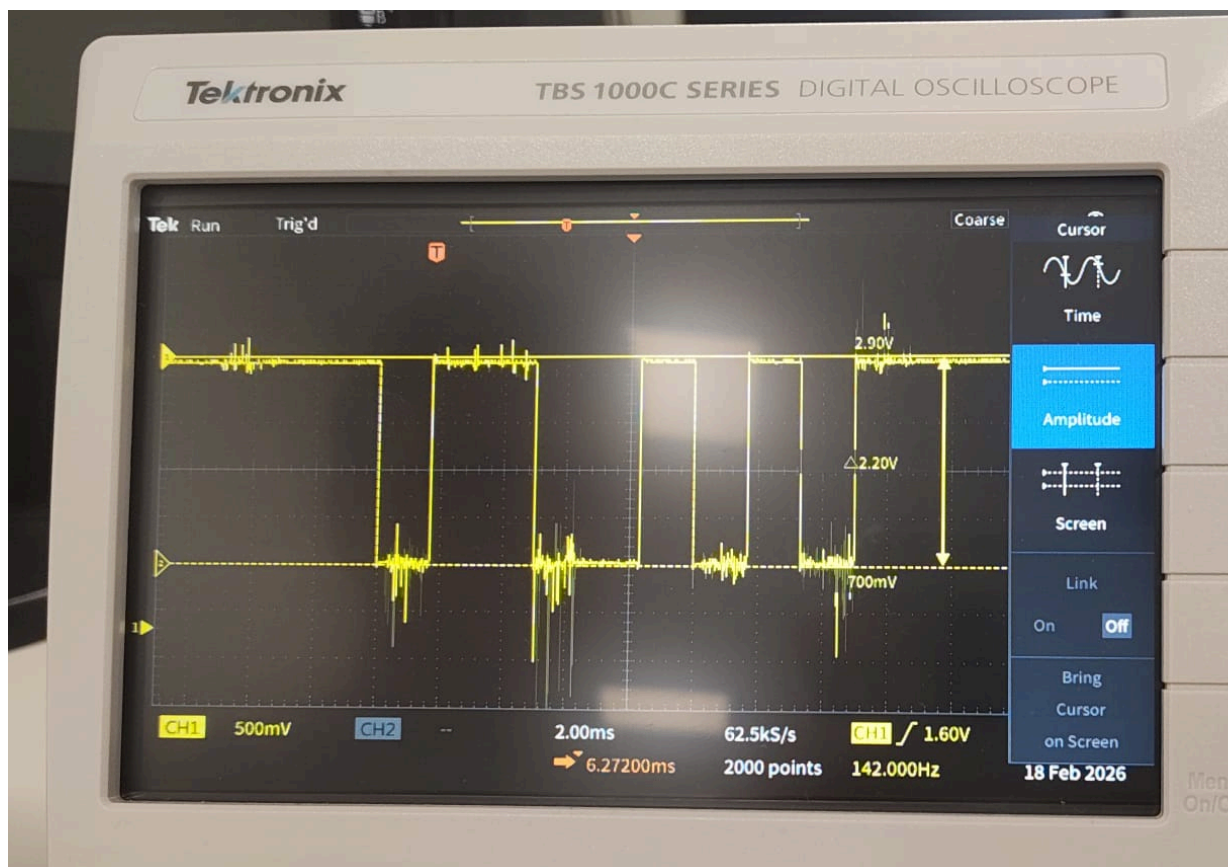
Se enviará (LSB → MSB): 11001010

Por tanto, la trama completa que se debería observar está formada por el bit de inicio (0), seguido de los datos en orden LSB a MSB (11001010), y finalmente el bit de parada (1). Esta secuencia coincide con la señal observada en el osciloscopio.

Con todo ello, se puede concluir que el experimento ha sido un éxito, ya que tanto la comunicación como la representación de la señal se corresponden con el comportamiento esperado.

3.3. Amplitud de la señal

El siguiente paso consistió en analizar la amplitud de la señal utilizando también el osciloscopio. Para ello, se emplearon las opciones de medida del propio equipo, lo que permitió observar con claridad los niveles de tensión.



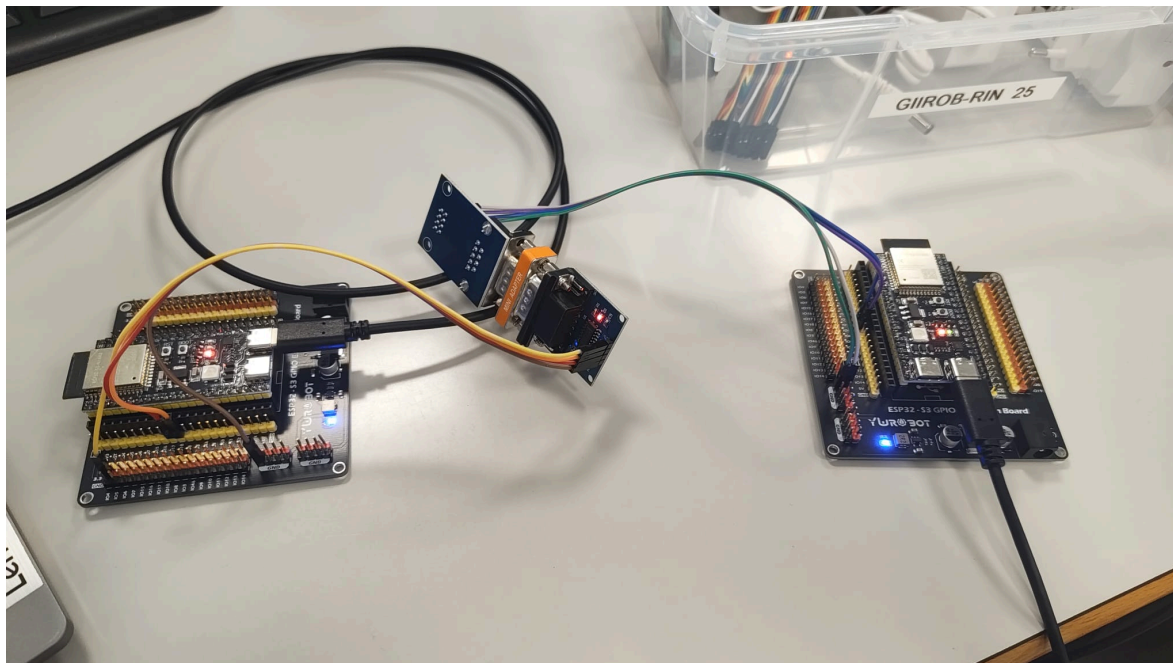
En la pantalla se identificó un nivel alto cercano a 2,90 V y un nivel bajo alrededor de 0,70 V. La diferencia entre ambos valores es de aproximadamente 2,20 V, lo que corresponde a la amplitud pico a pico de la señal.

En condiciones ideales, trabajando con una ESP32-S3-DevKitC-1, se esperaría un nivel bajo próximo a 0 V y un nivel alto cercano a 3,3 V. Sin embargo, las medidas obtenidas muestran cierta desviación respecto a estos valores.

Estas diferencias pueden explicarse por varios factores, como la influencia de la sonda del osciloscopio sobre el circuito, la posible atenuación si se está utilizando una sonda en modo x10, pequeñas caídas de tensión en el cableado, una referencia de masa no completamente estable o la presencia de ruido en la señal, que se aprecia especialmente en los flancos.

3.4. Cable NULL-Modem

En este apartado, en lugar de utilizar un cable RS232 convencional, se empleó un cable null modem. Para ello, fue necesario adaptar las conexiones, cruzando las líneas de transmisión y recepción, de manera que el pin de transmisión de una placa se conecta al de recepción de la otra, y viceversa, tal y como se muestra en la imagen.



Una vez realizado este cambio, se comprobó que la comunicación seguía funcionando correctamente, enviándose los datos de la misma forma que en la configuración anterior. Además, al observar la señal con el osciloscopio, se pudo verificar que esta coincidía con la obtenida previamente, confirmando así que el comportamiento del sistema se mantiene al utilizar el cable null modem.

Bibliografía

Material de apoyo a la docencia de PoliformaT.

OpenAI. (2026). ChatGPT (modelo de lenguaje).